

Empowering LLMs to Understand and Generate Complex Vector Graphics

Ximing Xing¹, Juncheng Hu¹, Guotao Liang¹, Jing Zhang¹, Dong Xu², Qian Yu^{1*}

¹Beihang University, China

{ximingxing, hujuncheng, liangguotao, zhang_jing, qianyu}@buaa.edu.cn

²The University of Hong Kong, China

dongxu@cs.hku.hk

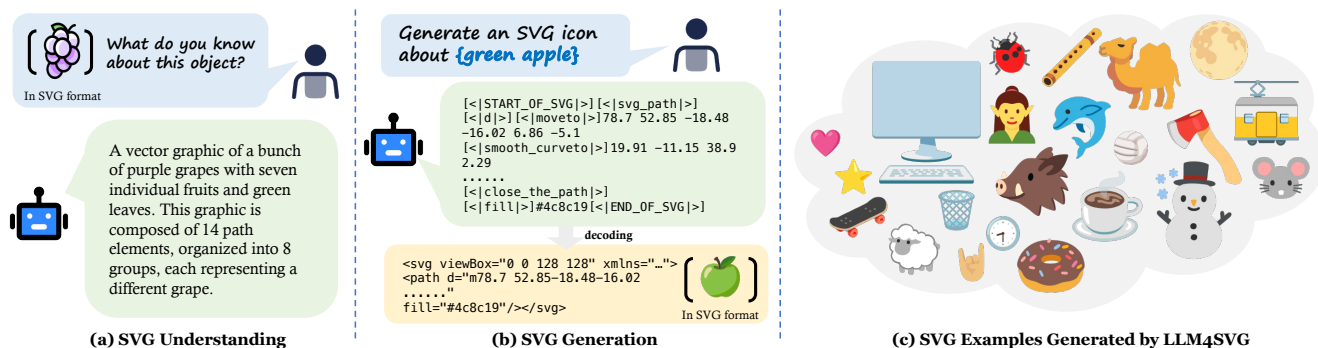


Figure 1. **Our LLM4SVG can understand and generate vector graphics from textual description.** Our LLM4SVG is designed to: (a) Understand the semantics of SVG (Scalable Vector Graphics) source code and directly extract the meanings conveyed by vector images; (b) Generate corresponding structured SVG representations from textual prompts and decode them into SVG source code that accurately reflects the described content. (c) illustrates some SVG examples generated by our method.

Abstract

The unprecedented advancements in Large Language Models (LLMs) have profoundly impacted natural language processing but have yet to fully embrace the realm of scalable vector graphics (SVG) generation. While LLMs encode partial knowledge of SVG data from web pages during training, recent findings suggest that semantically ambiguous and tokenized representations within LLMs may result in hallucinations in vector primitive predictions. Additionally, LLM training typically lacks modeling and understanding of the rendering sequence of vector paths, which can lead to occlusion between output vector primitives. In this paper, we present **LLM4SVG**, an initial yet substantial step toward bridging this gap by enabling LLMs to better understand and generate vector graphics. LLM4SVG facilitates a deeper understanding of SVG components through learnable semantic tokens, which precisely encode these tokens and their corresponding properties to generate semantically aligned SVG outputs. Using a series of learnable semantic tokens, a structured dataset for instruction following is developed to support comprehension and generation across

two primary tasks. Our method introduces a modular architecture to existing large language models, integrating semantic tags, vector instruction encoders, fine-tuned commands, and powerful LLMs to tightly combine geometric, appearance, and language information. To overcome the scarcity of SVG-text instruction data, we developed an automated data generation pipeline that collected our **SVGX-SFT Dataset**, consisting of high-quality human-designed SVGs and 580k SVG instruction following data specifically crafted for LLM training, which facilitated the adoption of the supervised fine-tuning strategy popular in LLM development. By exploring various training strategies, we developed LLM4SVG, which significantly moves beyond optimized rendering-based approaches and language-model-based baselines to achieve remarkable results in human evaluation tasks. Code, model, and data will be released at: <https://ximinng.github.io/LLM4SVGProject/>

1. Introduction

Scalable Vector Graphics (SVGs) constitute a fundamental image encoding paradigm wherein visual elements are constructed from primitive geometric entities defined by

*Corresponding author.

mathematical formulations, contrasting with raster graphics’ discrete pixel matrices. Vector-based representation offers resolution independence, preserving geometric precision across arbitrary scaling transformations without degradation. Vector graphics exhibit superior compression efficiency, optimizing storage requirements and transmission bandwidth. Their parametric editability enables precise manipulation of constituent elements—a characteristic instrumental during iterative design processes. These mathematical underpinnings render SVGs exceptionally suitable for applications demanding visual fluency and precision.

In recent years, there has been a significant increase in interest in vector graphics generation [14, 22, 25, 35, 43, 52, 56, 64, 65, 71]. Notwithstanding the significant advantages inherent to SVGs, current deep learning-based generative methods still face limitations in producing high-quality, complex SVG outputs. The current approaches [5, 18, 32, 42, 50, 59, 61] represent SVGs using a restricted command path and leverages sequential model learning. Such methods predominantly engage with simplified SVGs, confined to basic path commands (*e.g.* move to, line to, cubic bézier) and are frequently limited in complexity; certain approaches focus exclusively on fundamental fonts [32] or icons [59]. Recent innovations [25, 52, 64, 65, 71] have incorporated advanced image diffusion models [39, 44] to facilitate the generation of raster images, subsequently translated into SVG format via a differentiable rasterizer [30] predicated on bézier curve representations. While the utilization of generative raster images introduces a degree of variety, this process is characterized by an cumbersome iterative procedure, and the resultant SVGs remain non-editable and fail to align with the expectations of professional designers. In light of these developments, a critical gap persists in the realm of systems capable of directly synthesizing intricate and detailed SVG code, fully leveraging the comprehensive array of SVG primitives requisite for sophisticated design applications.

Recent advancements in large language models (LLMs) [2, 4, 31, 36, 54] have evidenced their capacity to comprehend and parse XML syntax [6], achieved through extensive pre-training on a diverse corpus of text data sourced from the Web. This proficiency establishes a robust foundation for LLMs to synthesize vector graphics [43]. Notably, state-of-the-art models, such as GPT-4 [2] and Claude [4], show proficiency in generating simple vector primitives like triangles and rectangles; however, they often encounter significant limitations in synthesizing complex graphics. This is because, during pre-training, SVG data from the internet is often embedded within web page code, requiring LLMs to parse layered languages, which renders SVG data less accessible amidst lengthy XML tags. These limitations manifest as confusion or hallucinations in vector path sequences, yielding semantically ambiguous graphics

and improperly encoded attributes.

In this paper, we present LLM4SVG, an initial yet substantial step toward bridging this gap by enabling LLMs to better understand and generate vector graphics, as illustrated in Fig. 1. Built on an existing LLM/MLLM, LLM4SVG maximizes the model’s potential for vector graphic synthesis. Our paper makes the following contributions:

- **LLM4SVG: The First Framework Supporting Arbitrary LLMs for SVG Tasks.** We present LLM4SVG, a framework enabling any LLM or MLLM to understand and generate SVGs effectively. Our approach addresses vector primitive prediction challenges through learnable semantic tokens, precisely encoding SVG components and properties. This bridges natural language processing and vector graphics, aligning outputs with human design principles.
- **Modular Architecture with Decoupled Vector Instructions and Parameters.** LLM4SVG enhances traditional LLM architectures by decoupling vector instructions from parameters. This integration enables comprehensive understanding of SVG elements, producing semantically consistent vector graphics by merging geometry, appearance, and linguistic information.
- **Development of SVGX-SFT Dataset.** We introduce the SVGX-SFT Dataset with high-quality human-designed SVGs and 580k instruction following data crafted for LLM training. This dataset supports our training strategy, enhancing model performance in SVG generation and establishing a foundation for future vector graphics research.

2. Related Work

2.1. Vector Graphics Generation

Scalable Vector Graphics (SVGs) provide a declarative format for visual concepts articulated through primitives. One approach to generating SVG content entails training a neural network to generate predefined SVG commands and attributes [5, 18, 32, 42, 50, 59, 61]. Neural networks designed for learning SVG representations typically include architectures such as RNNs [18, 42], VAEs [5, 32, 50], and Transformers [5, 59, 61]. The training of these networks is heavily dependent on datasets in vector form. However, the limited availability of large-scale vector datasets significantly constrains their generalization capability and their ability to synthesize intricate vector graphics.

Li *et al.* [30] introduce a differentiable rasterizer that bridges the vector graphics and raster image domains. While image generation methods that traditionally operate over vector graphics require a vector-based dataset, this approach for SVG generation [22, 33, 47–49, 53] involves directly optimizing the geometric and color param-

ters of SVG paths using the guidance of a pretrained vision-language model. Recent advances in visual text embedding contrastive language-image pre-training model (CLIP) [41] has enabled a number of successful methods [14, 35, 56] for synthesizing sketches. In contrast to CLIP, several methods [25, 64, 65] integrate diffusion models with differentiable rasterizers to achieve superior generation capabilities and enhanced image consistency. Moreover, recent studies [52, 71] combine optimization-based approaches with neural networks to learn vector representations, incorporating geometric constraints into vector graphics.

2.2. Vector Graphics Understanding

Recent research in vector graphics has advanced both recognition and evaluation methodologies. YOLAT [28] pioneered treating vector graphics recognition as a detection problem without rasterization, though it struggles with complex hierarchical structures and semantic relationships. YOLAT++ [12] addressed these limitations by introducing hierarchical recognition capabilities and a new chart-based dataset. Meanwhile, VGBench [74] developed a comprehensive benchmark evaluating LLMs on vector graphics understanding and generation across various formats, revealing that while LLMs show promise, they perform poorly on low-level formats like SVG. Despite these advances, existing approaches remain constrained—recognition models lack generation capabilities, evaluation frameworks don’t address fundamental representation issues, and none adequately capture the bidirectional relationship between natural language and vector graphics necessary for creative design workflows.

2.3. Large Language Models

Large Language Models (LLMs) have made significant strides in natural language understanding, demonstrating strong generalization and reasoning abilities through extensive pre-training on large-scale text corpora [1–4, 9, 26, 34, 36, 51, 54, 63, 68].

LLMs can be broadly categorized into two types. The first type serves as an interface for individual modality-specific models [17, 23, 38], eliminating the need for re-training but relying heavily on external model availability. The second type employs an end-to-end training approach, which can either train models from scratch using large-scale multi-modal datasets [2, 24] or fine-tune pre-trained LLMs for specific applications [31]. Our work follows the latter strategy, adapting pre-trained LLMs to generate and understand vector graphics while maintaining flexibility for multi-modal extensions.

LLMs naturally extend into multi-modal domains, enabling them to process and generate content beyond text. Multi-modal Large Language Models (MLLMs) leverage this capability to handle diverse modalities, including im-

ages [16, 24, 31, 57, 58, 73], audio [10, 23], motion [27, 70], and 3D point clouds [20, 66]. This adaptability makes LLMs a powerful foundation for expanding into vector graphics and other specialized tasks.

2.4. Instruction Tuning

In natural language processing, researchers have explored various methods [36, 37, 60] for instruction-tuning LLMs to improve their ability to follow natural language instructions and perform real-world tasks. This straightforward approach has been shown to significantly enhance LLMs’ zero-shot and few-shot generalization capabilities. With the rise of multimodal large language models, LLaVA [31] has leveraged visual instruction tuning with vision-language data, greatly improving open-source MLLMs’ performance on multimodal tasks.

Recently, studies have investigated fine-tuning LLMs with image embeddings to generate Scalable Vector Graphics (SVG) by treating SVG code as a text-based representation [43, 67]. While promising, these approaches often overlook the hierarchical and structured nature of SVG files, treating them merely as sequential text.

3. SVGX-SFT Dataset

A significant obstacle in developing an end-to-end LLM is the acquisition of large-scale instruction-following data, which is indispensable for representation learning, aligning latent spaces, and guiding models to align with human intent [31]. In the domain of vector graphics, this obstacle is particularly acute, as the high production cost and tagging challenges associated with vector graphics constrain current research to a limited scope of applications. These areas include simple human hand drawings [45, 69], fonts [32], and iconic graphics [5, 7].

To address these challenges, we manually collected approximately 250,000 colorful and complex vector graphics and developed a normalization process to ensure that the collected data conformed to a consistent standard, including uniform canvas size, relative coordinate systems, and representation. These high-quality vector datasets provided us with a solid foundation for the development of LLM4SVG. Additionally, inspired by the recent success of GPT models in text annotation tasks [15], we utilized BLIP [29] to annotate rasterized vector graphs and GPT-4 [2] for instruction-following data collection.

SVG Re-captioning. SVG data collected from the Internet often contains noise, and directly using it for learning can compromise the model’s potential representational accuracy. Approximately half of the data in an SVG file is redundant for visual rendering. This redundancy includes: (1) temporary data used by vector editing applications, (2) non-optimal structural representations of SVG, and (3) unused and invisible graphic elements. We propose an SVG

Type	Role	Content Template
# 1	SYSTEM USER ASSISTANT	You are a helpful assistant, please help me generate SVG $\langle /s \rangle$ Generate an SVG illustration from the given description: $\langle \text{prompt} \rangle \langle /s \rangle$ $\langle \text{SOV} \text{ Path MoveTo Coord LineTo Coord ... FILL RGB ... EOv} \rangle \langle /s \rangle$
# 2	SYSTEM USER ASSISTANT	You are a helpful assistant, please help me generate an SVG from this image and description. $\langle /s \rangle$ Refer to rendering image: $\langle \text{img} \rangle$ and generate SVG from the given description: $\langle \text{prompt} \rangle \langle /s \rangle$ $\langle \text{SOV} \text{ Path MoveTo Coord LineTo Coord ... FILL RGB ... EOv} \rangle \langle /s \rangle$
# 3	SYSTEM USER ASSISTANT	Attempt to identify this SVG $\langle /s \rangle$ The following is an SVG illustration: $\langle \text{SOV} \text{ Path}_1 \dots \text{Path}_n \text{ EOv} \rangle$ Text description of this SVG: $\langle \text{desc} \rangle \langle /s \rangle$
# 4	SYSTEM USER ASSISTANT	Describe this SVG based on its image representation $\langle /s \rangle$ The following is an SVG illustration: $\langle \text{SOV} \text{ Path}_1 \dots \text{Path}_n \text{ EOv} \rangle$ rendering result: $\langle \text{img} \rangle$ Text description of this SVG: $\langle \text{desc} \rangle$. This SVG contains $\langle n_{\text{paths}} \rangle$ primitives. $\langle /s \rangle$
# 5	SYSTEM USER ASSISTANT	Describe this SVG based on its image representation $\langle /s \rangle$ SVG group 1: $\langle \text{GROUP} \text{ Path}_1 \dots \text{Path}_n \text{ EOv} \rangle$ rendering result: $\langle \text{img} \rangle_1$ SVG group 2: $\langle \text{GROUP} \text{ Path}_1 \dots \text{Path}_n \text{ EOv} \rangle$ rendering result: $\langle \text{img} \rangle_2 \langle /s \rangle$ Text description of this SVG: $\langle \text{desc} \rangle$ The 1st SVG group contains $\langle n_{\text{paths}} \rangle_1$ primitives representing $\langle \text{desc} \rangle_1$ The 2nd SVG group contains $\langle n_{\text{paths}} \rangle_2$ primitives representing $\langle \text{desc} \rangle_2 \langle /s \rangle$

Table 1. **Instruction Following Template.** We developed five distinct instruction templates tailored for tasks in vector graphics generation and understanding. Specifically, Types #1 and #2 facilitate the generation task, while Types #3, #4, and #5 focus on the understanding task. $\langle \text{prompt} \rangle$ denotes a brief image caption generated via BLIP [29], $\langle n_{\text{paths}} \rangle_i$ represents the total number of primitives in group i , $\langle \text{desc} \rangle$ provides a detailed GPT-4 [2] generated description, and Token represents different types of SVG semantic tokens. Path_i serves as the representation of a complete SVG primitive, encompassing a structured set of SVG semantic tokens essential for comprehensive vector graphic description. Types #1~#5 provide a structured framework for training SVG semantic tokens, facilitating more accurate vector representation and understanding. Losses are computed only on model responses. $\langle /s \rangle$ indicates the end-of-sentence token. “SYSTEM” is an instruction that describes the type of task, specifically the context of the conversation. “ASSISTANT” denotes the output generated in response to the instruction, representing the LLM’s reply. “USER” refers to the input data provided by the user.

data preprocessing pipeline designed to losslessly reduce the size of SVG files generated by vector editing applications. Details are provided in Sec. B and Fig. S5 of Supplementary. After optimizing the SVG, we rasterize it into an image of 512×512 pixels and use the BLIP [29] model to generate a corresponding caption as a text prompt. Consequently, we obtain a multimodal dataset, each entry of which is a triplet consisting of the optimized SVG, its corresponding rasterized image, and a text description generated by the BLIP model.

Subsequently, we propose a strategy for the automatic generation of SVG instruction-following data. As outlined in Table 1, the instruction data are categorized into two distinct parts: the first (items #1 and #2) addresses the synthesis of vector graphics, while the second part (items #3, #4, #5) pertains to the comprehension of vector graphics.

SVG Instruction Following Data. The constructed dataset adheres to a standardized instruction format, as depicted in Table 1 #1, comprising Text-SVG pairs for fine-tuning in text-to-SVG generation tasks. For text-guided SVG synthesis, visual prompts are indispensable. As illustrated in Table 1 #2, Text-Image-SVG triples facilitate instruction tuning for text-and-image to SVG generation.

During the SVG re-captioning phase, we obtained the

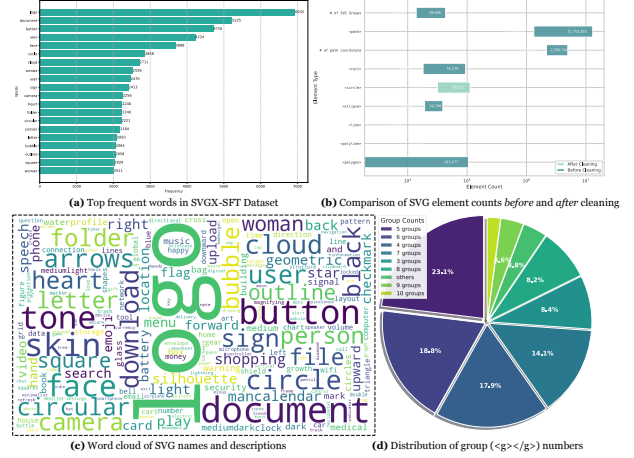


Figure 2. **Overview of SVGX-SFT Dataset.** (a) *Top Frequent Words in SVGX-SFT Dataset.* The most frequently occurring words in the SVGX-SFT dataset, highlighting common patterns and terminology used in SVG metadata. (b) *Comparison of SVG Element Counts Before and After Cleaning.* A comparison of the number of SVG elements before and after the cleaning process. The reduction in element count demonstrates the effectiveness of our preprocessing steps. (c) *Word Cloud of SVG Names and Descriptions.* A word cloud visualization of SVG names and descriptions, illustrating the distribution and emphasis of different terms in the dataset. (d) *Group (<g> </g>) Number Distribution.* The distribution of <g> (group) elements in the dataset, showing the frequency of grouped elements and their structural significance within SVG files.

corresponding text descriptions from the BLIP based on the rendering results, which were sufficient for text prompts, but too short for comments to understand SVG. Inspired by the recent success of GPT models in text annotation tasks [15], we utilized ChatGPT [36]/GPT-4 [2] for instruction-following data generation.

In total, we collected a dataset consisting of 250k annotated, high-quality, and standardized vector graphics, along with 580k unique SVG-Text-Image samples. The distribution across sample types is as follows: 250k samples of type #1, which extend to 250k samples of type #2, 60k samples of types #3 and #4, and 20k samples of type #5.

4. Instruction Tuning Scheme

We then delve into the architecture of LLM4SVG, which takes as input an SVG and user instruction and outputs responses. We first introduce the definition of a semantic token, and then introduce the two-stage training strategy.

4.1. SVG Semantic Tokens

For an input SVG X_v , we convert it from raw code into a structured representation. To accomplish this, we defined 55 SVG semantic tokens $s_i \in \mathbb{R}^{55}$ (including 15 tag tokens,

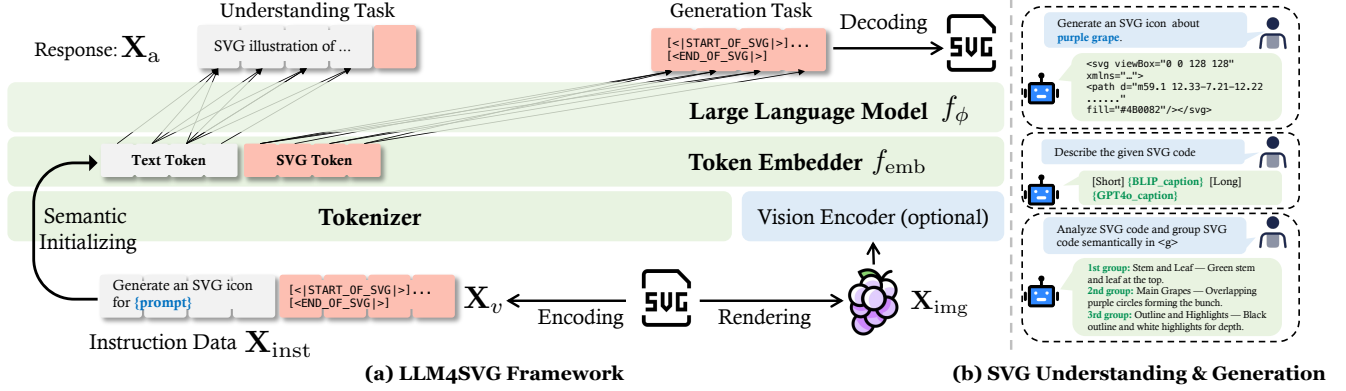


Figure 3. **An Overview of LLM4SVG.** Our LLM4SVG is capable of understanding and generating SVGs effectively. (1) During the training phase, we provide both the original SVG code $\mathbf{X}_v$ and the corresponding instruction data $\mathbf{X}_{\text{inst}}$ as input. For the understanding task, we use detailed descriptions $\mathbf{X}_a$ generated by GPT-4 [2] as the training labels. For the generation task, the SVG code portion is masked and serves as the target that the model needs to predict. (2) During the inference phase, for the understanding task, given an SVG source code, the model generates a description that aligns with the semantics expressed by the SVG. For the generation task, the model generates an SVG based on the input text prompt. During both training and inference phases, the rendered image $\mathbf{X}_{\text{img}}$ of the SVG can be used as conditional input to the model, guiding the content that the model understands or generates.

30 attribute tokens, 10 path command tokens, details are provided in Sec. C and Fig. S1 of Supplementary). These SVG tokens are used to replace all tags and attributes in the SVG source code, thus preventing the textual encoding of SVG tags and attributes as regular text. For example, the tag `<path>` will be tokenized as an SVG semantic token, rather than the literal “path” by the tokenizer. This ensures the uniqueness of SVG tags and attributes, and allows for their efficient integration into LLMs in a manner that is consistent with SVG definitions and optimizes token initialization. We adapt the embedding layer $\mathbf{W}_{\text{emb}} \in \mathbb{R}^{|\mathcal{V}'|}$ to learn the embeddings of these new tokens, where $|\mathcal{V}'| := |\mathcal{V}| + 55$ represents the sum of the size of the original vocabulary and the additional SVG tokens. Each new token is initialized based on the semantic average of its descriptive text s , as defined by the equation:

$$\mathbf{E}(s) = \frac{1}{n} \sum_{j=1}^n \mathbf{W}_{\text{emb}}^\top \cdot w_j \quad (1)$$

where w_j represents the j -th description token of s , $\mathbf{E}(\cdot)$ represents the token embedding layer and n represents the length of the token after encoding the description s . This initialization provides a good starting point for each SVG token and builds a compact, distributed representation for all SVG tokens.

4.2. Architecture

The primary goal is to effectively leverage the capabilities of both the pre-trained LLMs and visual models. The network architecture is illustrated in Fig. 3. We chose GPT-2 [40], Phi-2 [26, 34] and Falcon [3] as the foundational LLMs for our framework, denoted as f_ϕ and parameterized by ϕ . These models were chosen because they pos-

sess the ability to understand both visual and textual data, and they demonstrate effective instruction-following properties in various language tasks among existing open source models. Theoretically, other LLMs with similar capabilities could also serve as the bases for our method.

4.3. Training

For each SVG $\mathbf{X}_v$, we sample multi-turn conversation data $(\mathbf{X}_{\text{un}}^1, \mathbf{X}_{\text{gen}}^1, \dots, \mathbf{X}_{\text{un}}^T, \mathbf{X}_{\text{gen}}^T)$ from our SVG instruction dataset, where $\mathbf{X}_{\text{un}}$ represents the understanding tasks and $\mathbf{X}_{\text{gen}}$ refers to the generation tasks. T denotes the total number of turns in the conversation.

We apply instruction-tuning to the LLM using its original auto-regressive training objective for enhancing its performance on prediction tasks. Specifically, for a sequence of length L , we compute the probability of the target answers $\mathbf{X}_a$ using the following equation:

$$p(\mathbf{X}_a | \mathbf{X}_v, \mathbf{X}_{\text{inst}}) = \prod_{i=1}^L p_\theta(\mathbf{x}_i | \mathbf{X}_v, \mathbf{X}_{\text{inst}}, \mathbf{X}_a, \hat{\mathbf{x}}_{i-1}) \quad (2)$$

where θ represents the trainable parameters of the model, $\mathbf{X}_{\text{inst}}$ and $\mathbf{X}_a$ are the tokens corresponding to the instructions and answers for all preceding turns before the current prediction token $\mathbf{x}_i$, respectively. The term $\hat{\mathbf{x}}_{i-1} := (\mathbf{x}_{i-1}, \dots, \mathbf{x}_1)$ denotes the sequence of tokens that have been predicted in previous steps.

Stage 1: Pre-training for Feature Alignment. These pairs are converted to instruction-following data using the naive expansion method describe in Sec. 3. Each sample can be treated as a single-turn conversation. To construct the input $\mathbf{X}_{\text{inst}}$, instruction data is randomly sampled. In training, we keep the weights of both the visual encoder and the LLM frozen, and focus on maximizing the likelihood of Eq. 2

with trainable parameters $\theta = \mathbf{W}_{\text{emb}}$ only, where $\mathbf{W}_{\text{emb}}$ represents the word embedding.

Stage 2: Fine-tuning End-to-End. We employ the entire instruction dataset for supervised fine-tuning of all parameters, including those of the LLM, *i.e.*, the trainable parameters are $\theta = \{\mathbf{W}_{\text{emb}}, \phi\}$ in Eq. 2. We consider two specific use case scenarios:

- *Efficient parameters fine-tuning.* Methods like LoRA [11, 21] only fine-tune a small number of additional model parameters, significantly decreasing computational and storage costs while delivering performance comparable to that of a fully fine-tuned model. This approach facilitates the training and storage of LLMs on consumer hardware.
- *Full fine-tuning.* Fine-tuning all parameters requires higher computational resources, especially in large language models.

5. Experiments

Overview. This section outlines the core aspects of our model implementation, followed by a description of the dataset collection and preprocessing pipeline, which involves 250,000 high-quality SVG files, as detailed in Sec. 5.1. We then empirically evaluate the effectiveness of our model, LLM4SVG, in generating high-quality SVGs. This evaluation benchmarks our model against current state-of-the-art methods both qualitatively and quantitatively, as detailed in Sec. 5.2 and 5.3. This evaluation is augmented with a comprehensive architectural analysis, including ablation studies detailed in Sec. 5.4, to identify the specific contributions of individual model components to the overall performance.

Implementation Details. Our training process consists of two steps. In the first step, we add 55 SVG semantic tokens (Tab. S1) to the tokenizer and initialize the word embeddings $\mathbf{W}_{\text{emb}}$ using the semantic average of the description text as described in Eq. 1. In the second step, we perform supervised fine-tuning (SFT) using two alternative methods: either LoRA/QLoRA [11, 21] fine-tuning or full-parameter fine-tuning. The trainable parameters include the word embeddings and the parameters involved in SFT training. SFT training typically requires 1 to 3 epochs.

During the training process, we apply the AdamW optimizer with hyper-parameters $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 1 \times 10^{-8}$. We use a learning rate of $lr = 3 \times 10^{-4}$ with a cosine scheduler type and warmup ratio of 0.1. The training runs for 2 epochs with a maximum token length of 4096. If an SVG’s token sequence exceeds this length, it is directly truncated to the maximum length to ensure computational efficiency. Our implementation is based on LlamaFactory framework [72]. Considering VRAM limitations, we also integrated Unsloth [8] to support efficient training of quantized models. The training was performed using 8 NVIDIA A800 GPUs.

Evaluation Metrics. To facilitate a comprehensive assessment of our proposed method compared to baseline approaches, we classify all methods into three categories: Optimization-based methods, Neural Network-based methods, and LLM-based methods. We then evaluate these methods across two key dimensions: visual quality and computational cost. For visual quality, we measure (1) visual quality using FID (Fréchet Inception Distance) [19]; (2) text prompt alignment through the CLIP score [41]; and (3) aesthetic appeal using the Aesthetic score [46] and HPS (Human Preference Scores) [62]. (4) for computational cost, we test and compare the time cost of generating 10 SVGs by each method. (5) Avg.Tok represents the length of SVG code after removing comments and whitespace.

5.1. SVGX-SFT Dataset Analysis

Figure 2 provides a comprehensive breakdown of the SVGX-SFT dataset, which is designed for LLM4SVG through supervised fine-tuning. Fig. 2(a): The most common words in the dataset include “logo”, “document”, “button”, and “download”, suggesting that the dataset primarily consists of UI-related SVG elements. This indicates a strong presence of design-related components, making the dataset particularly useful for training models that need to understand or generate UI elements. Fig. 2(b): The dataset underwent a cleaning process, significantly reducing the number of `<path>` elements and path coordinates. This removes redundant or unnecessary elements, improving data quality and reducing complexity. However, certain elements like `<rect>`, `<circle>`, and `<ellipse>` remained relatively stable, indicating their importance in the dataset. Fig. 2(c): The word cloud visualization highlights key terms associated with SVG elements, reinforcing the dominant themes from subfigure (a). Words like “button”, “cloud”, “folder”, and “icon” emphasize the dataset’s focus on UI-related graphics. The presence of words such as “geometric”, “outline”, and “pattern” suggests that the dataset also includes a variety of structured vector graphics. Fig. 2(d): The pie chart shows the distribution of SVG group counts, with a significant portion (23.1%) having 5 groups, followed by other group counts such as 4, 6, and 3. This indicates that a considerable number of SVGs in the dataset are structured hierarchically, which is crucial for understanding compositional relationships in vector graphics.

5.2. Quantitative Evaluation

Table 2a presents a comparison of our approach with the most prominent text-to-SVG baseline methods across the previously defined dimensions. Our LLM4SVG model achieves the highest performance among LLM-based SVG generation methods. While it may not surpass optimization-based methods in terms of visual quality, it remains competitive, closely matching the performance of other meth-

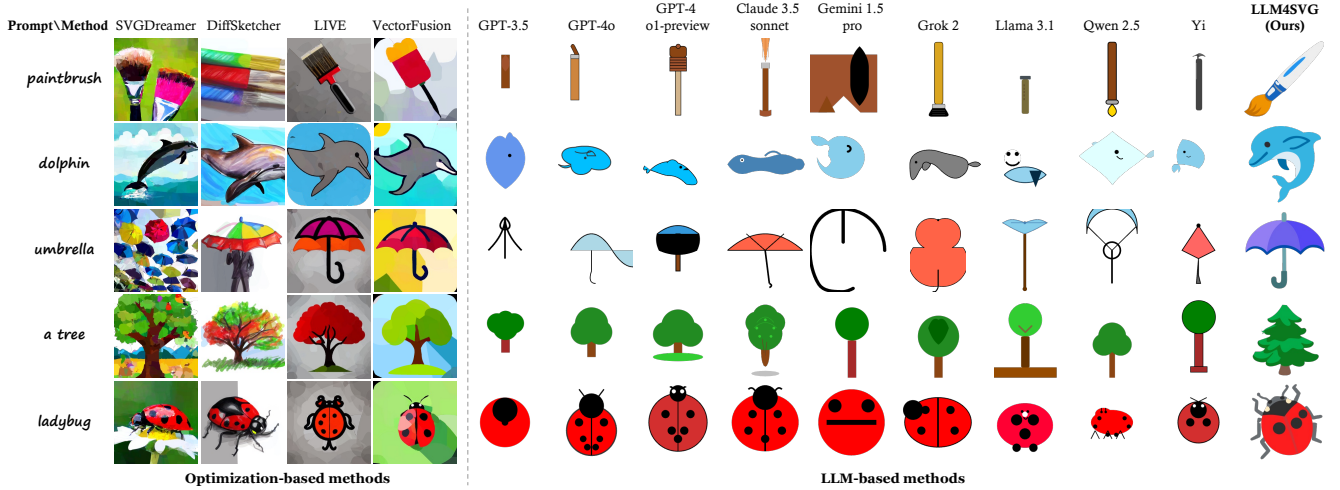


Figure 4. **Qualitative comparison of LLM4SVG with state-of-the-art SVG generation methods**, categorized into optimization-based and LLM-based approaches. The prompts (left) guide the generation of vector graphics across different methods. Optimization-based methods (left section) focus on refining SVGs through iterative optimization, while LLM-based methods (right section) generate SVGs directly from text descriptions. The results highlight differences in abstraction, structure, and fidelity to the input prompt, with our LLM4SVG achieving more structured and visually coherent outputs.

Method / Metric	Type	Visual Metric				Latency
		FID↓	CLIPScore↑	Aesthetic↑	HPS↑	
CLIPDraw [14]	Optim-based	132.75	0.2486	3.9803	0.2347	5min20s
Evolution [53]	Optim-based	123.97	0.1932	4.0845	0.1955	49min42s
DiffSketcher[64]	Optim-based	77.35	0.2402	4.1562	0.2423	12min9s
LIVE+VectorFusion [25]	Optim-based	84.71	0.2298	4.5165	0.2334	32min19s
VectorFusion [25]	Optim-based	87.73	0.2720	4.9845	0.2450	11min27s
SVGDreamer [65]	Optim-based	72.68	0.3001	5.5432	0.2685	43min56s
SVG-VAE [32]	NN-based	79.25	0.1893	2.674	0.098	1min4s
DeepSVG [5]	NN-based	71.37	0.2118	3.0017	0.109	2min3s
Iconshop [61]	NN-based	85.45	0.2489	3.4682	0.1376	1min8s
StrokeNUWA [50]	NN-based	92.31	0.3001	5.5432	0.1659	20s
LLM4SVG(GPT-2 small)	LLM-based	78.10	0.3129	5.7327	0.2076	12s
LLM4SVG(GPT-2 large)	LLM-based	66.09	0.3205	5.8729	0.2190	14s
LLM4SVG(GPT-2-XL)	LLM-based	64.11	0.3496	5.9836	0.2485	18s
LLM4SVG(Phi-2)	LLM-based	65.98	0.3373	5.9124	0.2289	20s
LLM4SVG(Falcon)	LLM-based	77.13	0.3018	4.9846	0.2012	25s
LLM4SVG(LLaVA)	LLM-based	66.72	0.3296	5.6846	0.2177	25s

(a) **Quantitative Comparison between LLM4SVG and State-of-the-Art Text-to-SVG Methods.**

Model	Input	FID↓	CLIPScore↑	Aesthetic↑	HPS↑	Avg.Tok
Llama-3.1 70B [13]	Text	138.44	0.2735	4.3048	0.1665	707.67
Gemini-1.5 Pro [9]	Text&Img	145.76	0.2622	4.2708	0.1511	547.50
Claude-3.5 [4]	Text	82.89	0.3083	5.2370	0.1912	736.38
Yi-1.5 34B [68]	Text	140.83	0.2824	4.5118	0.1676	633.42
Grok-2 [63]	Text	116.99	0.2840	4.8086	0.1663	581.88
Qwen2.5 70B [51]	Text	131.46	0.2803	4.5024	0.1691	705.00
GPT-3.5 [36]	Text&Img	129.40	0.2949	4.3070	0.1717	530.59
GPT-4o [2]	Text&Img	127.78	0.2949	5.0262	0.1788	654.12
GPT-4o1-preview [2]	Text&Img	135.33	0.2968	4.7754	0.1840	755.71
LLM4SVG(GPT-2 XL)	Text	64.11	0.3496	5.9836	0.2485	2297.75
LLM4SVG(Falcon)	Text	77.13	0.3018	4.9846	0.2012	1829.49
LLM4SVG(LLaVA)	Text&Img	66.72	0.3296	5.6846	0.2177	2009.41

(b) **Quantitative Comparison between LLM4SVG and State-of-the-Art LLMs.**

Table 2. **Quantitative Comparison of LLM4SVG.** (a) is used to indicate comparisons against SVG generation methods. (b) is employed to compare performance with LLM-based methods.

ods across several evaluation metrics. Furthermore, our approach only requires model inference, eliminating the need for a time-consuming optimization process, which results in a significantly shorter SVG generation time compared to optimization-based methods.

As shown in table 2b, our LLM4SVG outperforms all other LLMs across every aspect. Moreover, due to the limited support for continuous numerical data in most LLMs, SVGs generated by these models often exhibit imprecise coordinates and color representations, typically relying on integers rather than decimals and using basic color names like black or blue, instead of more precise hexadecimal codes. Our approach addresses these shortcomings by incorporating decimal coordinates and hexadecimal color codes for SVG paths, thereby expanding the model’s ability to represent a wider and more accurate range of colors (as il-

lustrated in Fig. S1). Further comparisons with LLM-based methods are provided in Sec. A of Supplementary.

5.3. Qualitative Evaluation

Figure 4 illustrates the visual quality of SVG generation methods, comparing both optimization-based and LLM-based approaches. It is evident that our method outperforms other LLM-based methods in terms of the completeness of the SVG generation, the selection and placement of primitives, and the semantic richness conveyed by the vector graphics. Optimization-based methods [14, 25, 53, 64, 65] use samples from the Latent Diffusion Model as supervision during the SVG generation process. Consequently, these methods normally employ a large number of overlapping and interwoven primitives to closely approximate the realistic samples. This often lead to excessive stroke redundancy,

Metric/Method	GPT4-o [2]	Claude-3.5 [4]	LLM4SVG	Human SVG
Prompt Alignment	0.49	0.56	0.89	0.94
Visual Quality	0.61	0.74	0.92	0.95
Pick Score	0.57	0.69	0.88	0.92

Table 3. **Results of Human Evaluation.** Human SVG refers to the SVGs designed by human designers in our collected dataset.

and the individual shapes of the primitives may appear irregular when viewed in isolation, making them less practical for use in real-life applications.

Human Evaluation. To evaluate the visual effects of our generated SVGs compared to other LLMs, we conducted a user study. As Claude 3.5 and GPT-4 are currently recognized as two of the best LLMs, we compared SVGs generated by our LLM4SVG with those produced by these models. In this study, we shuffled the SVGs generated by the different models together with the selected examples from our dataset. Participants were only provided with the text descriptions and the shuffled SVGs. They were then asked to evaluate the SVGs based on prompt alignment, visual quality, and their willingness to use the SVGs in real-world scenarios. The results are presented in Table 3, where our generated SVGs significantly outperformed other LLM-based methods, and the visual quality of our SVGs is closely comparable to that of SVGs created by humans. Further details of human evaluation are provided in Sec. D of Supplementary.

5.4. Ablation Study

Analysis of the LLM4SVG Architecture. The performance of LLM4SVG is significantly influenced by the underlying base model, particularly in how the tokenizer processes continuous numeric tokens. For instance, in the Qwen models [51, 57], all numbers and decimal points are tokenized as a single unit. This approach prevents Qwen from effectively handling continuous numerical data, resulting in poor performance in tasks such as coordinate prediction and hexadecimal color generation. Consequently, the model struggles to produce complete and coherent SVGs. In contrast, LLMs like GPT-2 [40], which explicitly enumerate all numbers up to 1,000 as individual tokens, demonstrate a stronger numerical understanding. This design enables them to generate more precise coordinates and color values, leading to more visually coherent and aesthetically refined SVGs.

Analysis of SVG Semantic Tokens. To enhance SVG understanding, we expanded the tokenizer with 55 SVG-specific semantic tokens, allowing the model to distinguish SVG tags and attributes from general text. For instance, without this modification, the word “path” could ambiguously refer to an SVG tag or a physical pathway. With a dedicated `<path>` token, the model can now differentiate between the two, improving SVG generation accuracy.

We analyze the impact of these tokens using t-SNE vi-

sualization in Sec. C of Supplementary. The green dots represent embeddings initialized with descriptions, forming structured clusters for SVG geometry and path command tags. In contrast, the orange crosses, which represent randomly initialized tokens, are scattered, leading to slower convergence. After training, as shown by the blue squares, the embeddings become well-organized, demonstrating that the model effectively learns semantic relationships between SVG elements.

6. Conclusion & Discussion

In this work, we introduced *LLM4SVG*, the first framework designed to support existing LLMs/MLLMs in both understanding and generating SVGs. Our approach allows language models to directly interpret SVG source code and produce high-quality SVGs that demonstrate meaningful complexity and align with human design principles. Our methodology employs a structured SVG encoding strategy that overcomes the limitations of LLMs treating SVG source code merely as plain text. Additionally, we developed the *SVGX-SFT Dataset*, which comprises high-quality SVGs created by human designers and dialogue-based instruction pairs tailored specifically for training LLMs. We anticipate that this dataset will significantly accelerate future research in vector graphics.

Future Work. LLM4SVG lays the foundation for language models in vector graphics, with several promising directions for future exploration. Enhancing semantic understanding of SVG elements could enable more meaningful editing and generation. Bridging text, raster images, and vector graphics through cross-modal translation would improve creative workflows. Supporting iterative refinement via natural language feedback could make SVG generation more interactive and user-friendly. Additionally, adapting the approach to specific domains like data visualization or UI/UX design could enhance its practical applications.

Acknowledgment. This work was supported in part by the National Science and Technology Major Project (No. 2022ZD0117800), the Young Elite Scientists Sponsorship Program by the Chinese Association for Science and Technology (CAST), and the Fundamental Research Funds for the Central Universities. It was also partially funded by the National Natural Science Foundation of China (No. 62461160331, No. 62132001), the Hong Kong Research Grants Council (RGC) General Research Fund (No. 17203023), and the Hong Kong RGC Collaborative Research Fund (No. C5052-23G). Additionally, this research received support from the National Natural Science Foundation of China/RGC Collaborative Research Scheme (No. CRS.HKU703/24), the Huawei-BUAA Joint Lab, and the Hong Kong Jockey Club Charities Trust under Grant No. 2022-0174.

References

- [1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024. 3
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 2, 3, 4, 5, 7, 8, 12, 13
- [3] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, M  rouane Debbah,   tienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, et al. The falcon series of open language models. *arXiv preprint arXiv:2311.16867*, 2023. 5
- [4] Anthropic. Claude 3.5 sonnet. <https://www.anthropic.com/news/claude-3-5-sonnet>, 2024. 2, 3, 7, 8, 12, 13
- [5] Alexandre Carlier, Martin Danelljan, Alexandre Alahi, and Radu Timofte. Deepsvg: A hierarchical generative network for vector graphics animation. *Advances in Neural Information Processing Systems (NeurIPS)*, 33:16351–16361, 2020. 2, 3, 7
- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. 2
- [7] Louis Clou  tre and Marc Demers. Figr: Few-shot image generation with reptile. *arXiv preprint arXiv:1901.02199*, 2019. 3
- [8] Michael Han Daniel Han and Unsloth team. Unsloth, 2023. 6
- [9] DeepMind. Gemini pro. <https://deepmind.google/technologies/gemini/pro/>, 2024. 3, 7, 12
- [10] Soham Deshmukh, Benjamin Elizalde, Rita Singh, and Huaming Wang. Pengi: An audio language model for audio tasks. In *Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS)*, 2023. 3
- [11] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems*, 36, 2024. 6
- [12] Shuguang Dou, Xinyang Jiang, Lu Liu, Lu Ying, Caihua Shan, Yifei Shen, Xuanyi Dong, Yun Wang, Dongsheng Li, and Cairong Zhao. Hierarchically recognizing vector graphics and a new chart-based vector graphics dataset. *IEEE Transactions on Pattern Analysis and Machine Intelligence (T-PAMI)*, 46(12):7556–7573, 2024. 3
- [13] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. 7, 12
- [14] Kevin Frans, Lisa Soros, and Olaf Witkowski. CLIPDraw: Exploring text-to-drawing synthesis through language-image encoders. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 2, 3, 7
- [15] Fabrizio Gilardi, Meysam Alizadeh, and Ma  l Kubli. Chatgpt outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30), 2023. 3, 4
- [16] Tao Gong, Chengqi Lyu, Shilong Zhang, Yudong Wang, Miao Zheng, Qianmengke Zhao, Kuikun Liu, Wenwei Zhang, Ping Luo, and Kai Chen. Multimodal-gpt: A vision and language model for dialogue with humans. *ArXiv*, abs/2305.04790, 2023. 3
- [17] Tanmay Gupta and Aniruddha Kembhavi. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14953–14962, 2023. 3
- [18] David Ha and Douglas Eck. A neural representation of sketch drawings. In *International Conference on Learning Representations (ICLR)*, 2018. 2
- [19] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems (NeurIPS)*, 30, 2017. 6
- [20] Yining Hong, Haoyu Zhen, Peihao Chen, Shuhong Zheng, Yilun Du, Zhenfang Chen, and Chuang Gan. 3d-LLM: Injecting the 3d world into large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. 3
- [21] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. 6
- [22] Teng Hu, Ran Yi, Baihong Qian, Jiangning Zhang, Paul L Rosin, and Yu-Kun Lai. Supersvg: Superpixel-based scalable vector graphics synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 24892–24901, 2024. 2
- [23] Rongjie Huang, Mingze Li, Dongchao Yang, Jiatong Shi, Xuankai Chang, Zhenhui Ye, Yuning Wu, Zhiqing Hong, Jiawei Huang, Jinglin Liu, Yi Ren, Zhou Zhao, and Shinji Watanabe. Audiogpt: Understanding and generating speech, music, sound, and talking head, 2023. 3
- [24] Shaohan Huang, Li Dong, Wenhui Wang, Yaru Hao, Saksham Singhal, Shuming Ma, Tengchao Lv, Lei Cui, Owais Khan Mohammed, Barun Patra, Qiang Liu, Kriti Agarwal, Zewen Chi, Johan Bjorck, Vishrav Chaudhary, Subhojit Som, Xia Song, and Furu Wei. Language is not all you need: Aligning perception with language models. In *Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS)*, 2023. 3
- [25] Ajay Jain, Amber Xie, and Pieter Abbeel. Vectorfusion: Text-to-svg by abstracting pixel-based diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. 2, 3, 7

- [26] Mojan Javaheripi, Sébastien Bubeck, Marah Abdin, Jyoti Aneja, Sebastien Bubeck, Caio César Teodoro Mendes, Weizhu Chen, Allie Del Giorno, Ronen Eldan, Sivakanth Gopi, et al. Phi-2: The surprising power of small language models. *Microsoft Research Blog*, 1:3, 2023. 3, 5
- [27] Biao Jiang, Xin Chen, Wen Liu, Jingyi Yu, Gang YU, and Tao Chen. MotionGPT: Human motion as a foreign language. In *Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS)*, 2023. 3
- [28] Xinyang Jiang, Lu Liu, Caihua Shan, Yifei Shen, Xuanyi Dong, and Dongsheng Li. Recognizing vector graphics without rasterization. In *Proceedings of the 35th International Conference on Neural Information Processing Systems (NeurIPS)*, Red Hook, NY, USA, 2021. Curran Associates Inc. 3
- [29] Junnan Li, Dongxu Li, Caiming Xiong, and Steven C. H. Hoi. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International Conference on Machine Learning (ICML)*, 2022. 3, 4, 13
- [30] Tzu-Mao Li, Michal Lukáč, Gharbi Michaël, and Jonathan Ragan-Kelley. Differentiable vector graphics rasterization for editing and learning. *ACM Transactions on Graphics (TOG)*, 39(6):193:1–193:15, 2020. 2
- [31] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 34892–34916, 2023. 2, 3
- [32] Raphael Gontijo Lopes, David Ha, Douglas Eck, and Jonathon Shlens. A learned representation for scalable vector graphics. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019. 2, 3, 7
- [33] Xu Ma, Yuqian Zhou, Xingqian Xu, Bin Sun, Valerii Filev, Nikita Orlov, Yun Fu, and Humphrey Shi. Towards layer-wise image vectorization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16314–16323, 2022. 2
- [34] microsoft phi2 team. Phi-2: The surprising power of small language models. https://nips.cc/media/neurips-2023/Slides/83968_5GxuY2z.pdf, 2023. 3, 5
- [35] Piotr Mirowski, Dylan Banarse, Mateusz Malinowski, Simon Osindero, and Chrisantha Fernando. Clip-clop: Clip-guided collage and photomontage. *arXiv preprint arXiv:2205.03146*, 2022. 2, 3
- [36] OpenAI. Introducing chatgpt. <https://openai.com/index/chatgpt/>, 2023. 2, 3, 4, 7, 12
- [37] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022. 3
- [38] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023. 3
- [39] Ben Poole, Ajay Jain, Jonathan T. Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. 2
- [40] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019. 5, 8
- [41] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763. PMLR, 2021. 3, 6
- [42] Pradyumna Reddy, Michael Gharbi, Michal Lukac, and Niloy J Mitra. Im2vec: Synthesizing vector graphics without vector supervision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7342–7351, 2021. 2
- [43] Juan A. Rodriguez, Shubham Agarwal, Issam H. Laradji, Pau Rodriguez, David Vazquez, Christopher Pal, and Marco Pedersoli. Starvector: Generating scalable vector graphics code from images, 2023. 2, 3
- [44] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10684–10695, 2022. 2
- [45] Patsorn Sangkloy, Nathan Burnell, Cusuh Ham, and James Hays. The sketchy database: learning to retrieve badly drawn bunnies. *ACM Trans. Graph.*, 35(4), 2016. 3
- [46] Christoph Schuhmann. Improved aesthetic predictor. <https://github.com/christophschuhmann/improved-aesthetic-predictor>, 2022. 6
- [47] I-Chao Shen and Bing-Yu Chen. Clipgen: A deep generative model for clipart vectorization and synthesis. *IEEE Transactions on Visualization and Computer Graphics*, 28(12):4211–4224, 2022. 2
- [48] Yiren Song, Xuning Shao, Kang Chen, Weidong Zhang, Zhongliang Jing, and Minzhe Li. Clipvg: Text-guided image manipulation using differentiable vector graphics. In *Proceedings of the Conference on Artificial Intelligence (AAAI)*, 2023.
- [49] Hao Su, Xuefeng Liu, Jianwei Niu, Jiahe Cui, Ji Wan, Xinghao Wu, and Nana Wang. Marvel: Raster gray-level manga vectorization via primitive-wise deep reinforcement learning. *IEEE Transactions on Circuits and Systems for Video Technology (T-CSVT)*, 2023. 2
- [50] Zecheng Tang, Chenfei Wu, Zekai Zhang, Mingheng Ni, Shengming Yin, Yu Liu, Zhengyuan Yang, Lijuan Wang, Zicheng Liu, Juntao Li, et al. Strokenuwa: Tokenizing strokes for vector graphic synthesis. *arXiv preprint arXiv:2401.17093*, 2024. 2, 7
- [51] Qwen Team. Qwen2.5: A party of foundation models, 2024. 3, 7, 8, 12
- [52] Vikas Thamizharasan, Difan Liu, Matthew Fisher, Nanxuan Zhao, Evangelos Kalogerakis, and Michal Lukac. Nivel: Neural implicit vector layers for text-to-vector generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4589–4597, 2024. 2, 3

- [53] Yingtao Tian and David Ha. Modern evolution strategies for creativity: Fitting concrete images and abstract concepts. In *Artificial Intelligence in Music, Sound, Art and Design*, pages 275–291. Springer, 2022. 2, 7
- [54] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *ArXiv*, abs/2302.13971, 2023. 2, 3, 12
- [55] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9 (11), 2008. 13
- [56] Yael Vinker, Ehsan Pajouheshgar, Jessica Y Bo, Roman Christian Bachmann, Amit Haim Bermano, Daniel Cohen-Or, Amir Zamir, and Ariel Shamir. Clipasso: Semantically-aware object sketching. *ACM Transactions on Graphics (TOG)*, 41(4):1–11, 2022. 2, 3
- [57] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 3, 8, 12
- [58] Wenhai Wang, Zhe Chen, Xiaokang Chen, Jiannan Wu, Xizhou Zhu, Gang Zeng, Ping Luo, Tong Lu, Jie Zhou, Yu Qiao, and Jifeng Dai. VisionLLM: Large language model is also an open-ended decoder for vision-centric tasks. In *Thirty-seventh Conference on Neural Information Processing Systems (NeurIPS)*, 2023. 3
- [59] Yizhi Wang and Zhouhui Lian. Deepvecfont: Synthesizing high-quality vector fonts via dual-modality learning. *ACM Transactions on Graphics (TOG)*, 40(6), 2021. 2
- [60] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560*, 2022. 3
- [61] Ronghuan Wu, Wanchao Su, Kede Ma, and Jing Liao. Iconshop: Text-based vector icon synthesis with autoregressive transformers. *arXiv preprint arXiv:2304.14400*, 2023. 2, 7
- [62] Xiaoshi Wu, Keqiang Sun, Feng Zhu, Rui Zhao, and Hongsheng Li. Human preference score: Better aligning text-to-image models with human preference. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 2096–2105, 2023. 6
- [63] xAI team. Grok-2 beta release. <https://x.ai/blog/grok-2>, 2024. 3, 7, 12
- [64] Ximing Xing, Chuang Wang, Haitao Zhou, Jing Zhang, Qian Yu, and Dong Xu. Diffsketcher: Text guided vector sketch synthesis through latent diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. 2, 3, 7
- [65] Ximing Xing, Haitao Zhou, Chuang Wang, Jing Zhang, Dong Xu, and Qian Yu. Svgdreamer: Text guided svg generation with diffusion model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4546–4555, 2024. 2, 3, 7
- [66] Runsen Xu, Xiaolong Wang, Tai Wang, Yilun Chen, Jiangmiao Pang, and Dahua Lin. Pointllm: Empowering large language models to understand point clouds. In *ECCV*, 2024. 3
- [67] Zhongzheng Xu and Emily Wall. Exploring the capability of llms in performing low-level visual analytic tasks on svg data visualizations. *arXiv preprint arXiv:2404.19097*, 2024. 3
- [68] Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Heng Li, Jiangcheng Zhu, Jianqun Chen, Jing Chang, et al. Yi: Open foundation models by 01. ai. *arXiv preprint arXiv:2403.04652*, 2024. 3, 7, 12
- [69] Qian Yu, Feng Liu, Yi-Zhe Song, Tao Xiang, Timothy M Hospedales, and Chen Change Loy. Sketch me that shoe. In *Computer Vision and Pattern Recognition (CVPR)*, pages 799–807. IEEE, 2016. 3
- [70] Hang Zhang, Xin Li, and Lidong Bing. Video-LLaMA: An instruction-tuned audio-visual language model for video understanding. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 543–553, Singapore, 2023. 3
- [71] Peiying Zhang, Nanxuan Zhao, and Jing Liao. Text-to-vector generation with neural path representation. *arXiv preprint arXiv:2405.10317*, 2024. 2, 3
- [72] Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, and Zheyang Luo. LlamaFactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 400–410, Bangkok, Thailand, 2024. Association for Computational Linguistics. 6
- [73] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. MiniGPT-4: Enhancing vision-language understanding with advanced large language models. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. 3
- [74] Bocheng Zou, Mu Cai, Jianrui Zhang, and Yong Jae Lee. VGBench: Evaluating large language models on vector graphics understanding and generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3647–3659, Miami, Florida, USA, 2024. Association for Computational Linguistics. 3